

Phase 8 — Checkout & Orders Implementation Report

AL-AZHARI LIBRARY backend **Phase 8 (Checkout & Orders)** has been implemented, verified, and integrated into the project while preserving all Phase 0–7 baselines.

1. Verification Summary

Previous Baseline:

Test Suites: 65 passed, 65 total
Tests: 465 passed, 465 total
Failures: 0

New Phase 8 Suites:

+ tests/unit/order-schema.test.ts
+ tests/unit/order-state-machine.test.ts
+ tests/unit/orders-migration.test.ts
+ tests/unit/shipping-service.test.ts
+ tests/integration/order-checkout.test.ts
+ tests/integration/order-lifecycle.test.ts
+ tests/integration/order-concurrency.test.ts
+ tests/api/order-api.test.ts
+ tests/api/order-admin-api.test.ts

Final Verification Numbers:

Test Suites: 74 passed, 74 total
Tests: 516 passed, 516 total
Failures: 0
Regressions: 0
Typecheck: PASS (tsc --noEmit: 0 errors)
Lint: PASS (eslint: 0 warnings, 0 errors)
Build: PASS (tsc: clean dist output)

2. Architecture & Domain Boundaries

The orders domain is located under `src/modules/orders/` and `src/modules/shipping/`, respecting the modular monolith boundaries:

- **Orders Module (`orders`)**: Owns the authoritative order aggregate, immutable order line snapshots, public order references, order lifecycle state transitions, and owner-scoped idempotency.
- **Cart Module (`carts`)**: Cart lines and cart versioning remain owned by Cart domain; cleared atomically only upon successful order commit within the MongoDB transaction.
- **Catalog Module (`products`)**: Queried transactionally for current price and availability revalidation; not mutated by Orders.
- **Shipping Module (`shipping`)**: Configurable hierarchical shipping rules (`area > city > governorate > default` , resolved by highest priority) and pickup configuration (0 EGP).
- **Inventory Module (`inventory`)**: **Stock reservation is NOT performed on checkout.** Order creation leaves stock unchanged. Admin acceptance invokes the Phase 7 `reserveOrderStock()` application service. Order fulfillment (`delivered / picked_up`) invokes `deductOrderReservations()` .
- **Audit Module (`audit`)**: Records append-only audit events (`order.created` , `order.updated` , `order.cancelled` , `order.accepted` , `order.rejected` , `order.status_updated` , `order.shipping_updated`) without logging raw guest tokens, credentials, or PII.

3. API Surface Implemented

Public & Customer Endpoints:

- `POST /api/v1/checkout/shipping-estimate` — Hierarchical shipping estimation for delivery and pickup.
- `POST /api/v1/orders` — Authoritative checkout for registered customers and guests with idempotency and transaction safety.
- `GET /api/v1/orders/:reference` — Order retrieval with strict customer ownership and guest token (`X-Guest-Token`) access validation.
- `PATCH /api/v1/orders/:reference` — Customer edit of contact, fulfillment, or lines while order is strictly in `pending_review` with optimistic version checking.
- `POST /api/v1/orders/:reference/cancel` — Customer order cancellation while in `pending_review`.
- `POST /api/v1/orders/:reference/confirm-cod` — Customer confirmation for COD orders in `customer_confirmation_required`.

Admin Endpoints:

- `GET /api/v1/admin/orders` — Paginated order list with envelope metadata and status filters (`orders.read`).
- `GET /api/v1/admin/orders/:reference` — Detailed order view for administrators (`orders.read`).
- `POST /api/v1/admin/orders/:reference/accept` — Admin order acceptance with Phase 7 atomic stock reservation (`orders.accept`).
- `POST /api/v1/admin/orders/:reference/reject` — Admin rejection with defensive reservation release (`orders.write`).
- `POST /api/v1/admin/orders/:reference/status` — Operational order transitions (`preparing` , `shipped` , `out_for_delivery` , `delivered` , `ready_for_pickup` , `picked_up` , `completed`) with automated stock deduction upon delivery (`orders.write`).
- `POST /api/v1/admin/orders/:reference/shipping` — Admin carrier assignment and final shipping cost confirmation (`orders.write`).

4. Key Subsystem Implementations

1. State Machine Separation:

- Order Status (`order.status`): `pending_review` → `accepted` → (`awaiting_payment` | `customer_confirmation_required`) → (`confirmed` | `payment_confirmed`) → `preparing` → (`ready_for_pickup` | `shipped`) → (`picked_up` | `delivered`) → `completed` (or terminal `rejected` , `cancelled` , `returned`).
- Payment Status (`order.paymentStatus`): `Independent` (`not_submitted` , `proof_uploaded` , `under_review` , `confirmed` , `rejected` , `new_proof_requested`).
- Fulfillment Status (`order.fulfillment.shippingStatus`): Operational delivery/pickup state.

2. Guest Security:

- 256-bit cryptographically secure token generated (`crypto.randomBytes(32)`).
- Plaintext token returned **only once** at creation; only its SHA-256 hash is persisted in MongoDB (`guestAccessTokenHash`).
- Rejects unauthorized or cross-guest requests with HTTP 403.

3. Idempotency:

- Owner-scoped key with SHA-256 canonical request fingerprinting. Replay with identical payload returns the existing order; altered payload throws `IDEMPOTENCY_KEY_REUSED` (HTTP 409).

4. Price & Availability Revalidation:

- Re-reads catalog values inside the transaction. If price changed, rejects with `PRICE_CHANGED` (HTTP 409) and leaves cart untouched. If availability changed or out-of-stock, rejects with `AVAILABILITY_CHANGED` (HTTP 422) and preserves cart.

5. Database Migration:

- `20260928_006_orders.migration.ts` creates `orders` and `shippingRules` collections, unique reference index, customer history index, status index, idempotency partial unique index, and priority indexes. Fully registered and idempotent.

6. Documentation:

- Available in `docs/PHASE-8-CHECKOUT-ORDERS.md` and `Back-end/docs/PHASE-8-CHECKOUT-ORDERS.md`.

Phase 8 complete. Implementation stopped here. No Phase 9 Payments work was implemented.